

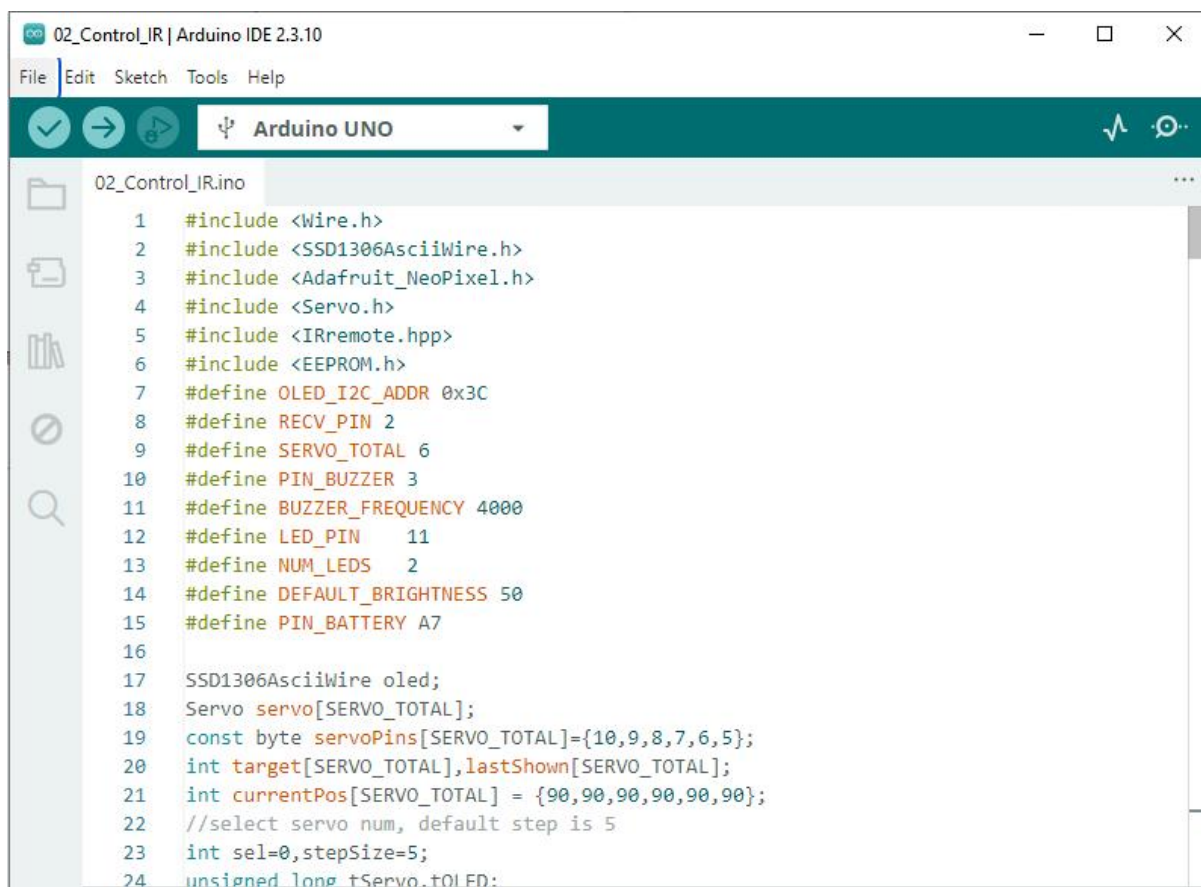
2 Controlling the Robot via IR

2.1 Overview

In this lesson, we will learn to control the robotic arm via an infrared remote controller, including functions such as arm movement, movement speed adjustment, step length switching, and posture recording.

2.2 Upload infrared control program

1. Connect your computer and Adeept Robotic Arm Board with a USB cable. Then connect the battery and turn on the power switch.
2. Open "2_Control_IR" folder in "/Code", double-click "2_Control_IR.ino".



```
02_Control_IR | Arduino IDE 2.3.10
File Edit Sketch Tools Help
Arduino UNO
02_Control_IR.ino
1 #include <Wire.h>
2 #include <SSD1306AsciiWire.h>
3 #include <Adafruit_NeoPixel.h>
4 #include <Servo.h>
5 #include <IRremote.hpp>
6 #include <EEPROM.h>
7 #define OLED_I2C_ADDR 0x3C
8 #define RECV_PIN 2
9 #define SERVO_TOTAL 6
10 #define PIN_BUZZER 3
11 #define BUZZER_FREQUENCY 4000
12 #define LED_PIN 11
13 #define NUM_LEDS 2
14 #define DEFAULT_BRIGHTNESS 50
15 #define PIN_BATTERY A7
16
17 SSD1306AsciiWire oled;
18 Servo servo[SERVO_TOTAL];
19 const byte servoPins[SERVO_TOTAL]={10,9,8,7,6,5};
20 int target[SERVO_TOTAL],lastShown[SERVO_TOTAL];
21 int currentPos[SERVO_TOTAL] = {90,90,90,90,90,90};
22 //select servo num, default step is 5
23 int sel=0,stepSize=5;
24 unsigned long tServo,tOLED;
```

3. Select development board and serial port.

Board: **Tools---**>**Board---**>**Arduino AVR Boards---**>**Arduino Uno**

Port: **Tools --->Port---**>**COMx**

Note: The port number will be different in different computers.



4. After opening, click  to upload the code program to the Arduino. If there is no error warning in the console below, it means that the Upload is successful.

```

Output
Sketch uses 2952 bytes (9%) of program storage space. Maximum is 32256 bytes.
Global variables use 82 bytes (4%) of dynamic memory, leaving 1966 bytes for local variables. Maximum is 2048 bytes.

```

2.3 Remote Control Functions Introduction

The corresponding relationship between the buttons of the infrared remote control and the functions of the car is as follows:

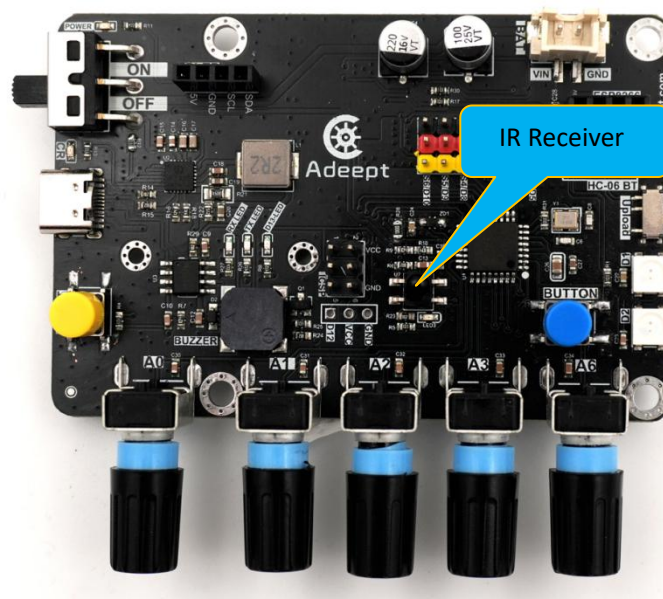
Button	Function	Button	Function
	The servo rotates toward 180 degrees	A	Switch the step size to 1 degree
	The servo rotates toward 0 degrees	B	Switch the step size to 3 degree
	\	C	Switch the step size to 5 degree
	\	D	Switch the step size to 10 degree
	Set the current servo to 90 degrees	E	Save the current posture. The robotic arm will stay in

			this posture every time it powers on.
0	Set all servos to 90 degrees.	F	Turn off the WS2812 LED lights.
1	Set S1 as the currently controlled servo.	2	Set S2 as the currently controlled servo.
3	Set S3 as the currently controlled servo.	4	Set S4 as the currently controlled servo.
5	Set S5 as the currently controlled servo.	6	Set S6 as the currently controlled servo.
7	Set the WS2812 LEDs to red.	8	Set the WS2812 LEDs to green.
9	Set the WS2812 LEDs to blue.		\

2.4 Controlling the Robot Arm with a Remote Control

Now that we have uploaded the infrared control program, you can operate the robotic arm according to the command instructions above. Please note that infrared control has distance limitations - when pressing remote control buttons, make sure to aim directly at the infrared receiver at close range.





2.5 Code

```

001 #include <Wire.h>
002 #include <SSD1306AsciiWire.h>
003 #include <Adafruit_NeoPixel.h>
004 #include <Servo.h>
005 #include <IRremote.hpp>
006 #include <EEPROM.h>
007 #define OLED_I2C_ADDR 0x3C
008 #define RECV_PIN 2
009 #define SERVO_TOTAL 6
010 #define PIN_BUZZER 3
011 #define BUZZER_FREQUENCY 4000
012 #define LED_PIN 11
013 #define NUM_LEDS 2
014 #define DEFAULT_BRIGHTNESS 50
015 #define PIN_BATTERY A7
016
017 SSD1306AsciiWire oled;
018 Servo servo[SERVO_TOTAL];
019 const byte servoPins[SERVO_TOTAL]={10,9,8,7,6,5};
020 int target[SERVO_TOTAL],lastShown[SERVO_TOTAL];
021 int currentPos[SERVO_TOTAL] = {90,90,90,90,90,90};
022 //select servo num, default step is 5
023 int sel=0,stepSize=5;
024 unsigned long tServo,tOLED;
025 Adafruit_NeoPixel pixels(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);
026 uint8_t wsMode=0;
027 uint32_t wsTimer=0;
028 const float ADC_REF_VOLTAGE = 5.1;
029 const float VOLTAGE_DIV_RATIO = 2.0;
030 const float BAT_MAX_VOLTAGE = 8.4;
031 float batteryVoltage=0;

```

```
032 int batteryADC=0;
033 float lastBatteryVoltage=0;
034 unsigned long oledTimer=0;
035 unsigned long batteryTimer=0;
036
037 void Buzzer_Setup(){
038     pinMode(PIN_BUZZER, OUTPUT);
039     digitalWrite(PIN_BUZZER, LOW);
040 }
041
042 void softTone(int durationMs){
043     unsigned long start = millis();
044     int delayUs = 500000 / BUZZER_FREQUENCY; // 半周期
045     while (millis() - start < durationMs) {
046         digitalWrite(PIN_BUZZER, HIGH);
047         delayMicroseconds(delayUs);
048         digitalWrite(PIN_BUZZER, LOW);
049         delayMicroseconds(delayUs);
050     }
051 }
052
053 void Buzzer_Alert(int beat, int rebeat){
054     beat = constrain(beat, 1, 9);
055     rebeat = constrain(rebeat, 1, 255);
056     for (int j = 0; j < rebeat; j++) {
057         for (int i = 0; i < beat; i++) {
058             softTone(100);
059             delay(100);
060         }
061         delay(500);
062     }
063 }
064
065 void saveEEP(){
066     for(int i=0;i<SERVO_TOTAL;i++){
067         EEPROM.update(i,target[i]);
068     }
069 }
070
071 void loadEEP(){
072     for(int i=0;i<SERVO_TOTAL;i++){
073         int v=EEPROM.read(i);
074         if(v>180)v=90;
075         target[i]=v;
076         lastShown[i]=-1;
077     }
078 }
079
080 void drawStatic(){
081     oled.clear();
082     oled.setFont(Adafruit5x7);
083     oled.setCursor(15,0);
084     oled.print("IR Control");
085     oled.setCursor(70,5);
086     oled.print("Step:");
087     for(int i=0;i<SERVO_TOTAL;i++){
```

```
088     oled.setCursor(0,i+2);
089     oled.print(" S");
090     oled.print(6-i);
091     oled.print(":");
092 }
093 }
094
095 void updateOLED(){
096     if(millis()-tOLED<1000)return;
097     tOLED=millis();
098     oled.setCursor(0,2+sel);
099     oled.print("#");
100     for(int i=0;i<SERVO_TOTAL;i++){
101         if(i!=sel){
102             oled.setCursor(0,i+2);
103             oled.print(" ");
104         }
105         if(lastShown[i]!=target[i]){
106             oled.setCursor(24,i+2);
107             oled.print(" ");
108             oled.setCursor(24,i+2);
109             oled.print(target[i]);
110             lastShown[i]=target[i];
111         }
112     }
113     oled.setCursor(100,5);
114     oled.print(" ");
115     oled.setCursor(100,5);
116     oled.print(stepSize);
117
118     if(lastBatteryVoltage != batteryVoltage){
119         oled.setCursor(96,0);
120         oled.print(" ");
121         oled.setCursor(96,0);
122         oled.print(batteryVoltage,2);
123         oled.print("V");
124         lastBatteryVoltage = batteryVoltage;
125     }
126 }
127
128 void updateServo(){
129     if(millis()-tServo<5)return;
130     tServo=millis();
131     for(int i=0;i<SERVO_TOTAL;i++){
132         if(currentPos[i]<target[i]) currentPos[i]++;
133         else if(currentPos[i]>target[i]) currentPos[i]--;
134         servo[i].write(currentPos[i]);
135     }
136 }
137
138 void initWS2812(){
139     pixels.begin();
140     pixels.clear();
141     pixels.show();
142 }
143
```

```

144 void setAllLED(uint8_t R, uint8_t G, uint8_t B, uint8_t brightness) {
145     uint8_t r = map(brightness, 0, 255, 0, R);
146     uint8_t g = map(brightness, 0, 255, 0, G);
147     uint8_t b = map(brightness, 0, 255, 0, B);
148     for (int i = 0; i < NUM_LEDS; i++) {
149         pixels.setPixelColor(i, pixels.Color(r, g, b));
150     }
151     pixels.show();
152 }
153
154 void updateWS2812(){
155     if(millis()-wsTimer<500) return;
156     wsTimer=millis();
157     switch(wsMode){
158         case 0:
159             setAllLED(0,0,0,DEFAULT_BRIGHTNESS);
160             break;
161         case 1:
162             setAllLED(255,0,0,DEFAULT_BRIGHTNESS);
163             break;
164         case 2:
165             setAllLED(0,255,0,DEFAULT_BRIGHTNESS);
166             break;
167         case 3:
168             setAllLED(0,0,255,DEFAULT_BRIGHTNESS);
169             break;
170     }
171 }
172
173 int Get_Battery_Voltage_ADC(){
174     int adcValue = 0;
175     for (int i = 0; i < 5; i++){
176         adcValue += analogRead(PIN_BATTERY);
177         delayMicroseconds(200);
178     }
179     return adcValue / 5;
180 }
181
182 void updateBattery(){
183     if (millis() - batteryTimer < 1000)return;
184     batteryTimer = millis();
185     batteryADC = Get_Battery_Voltage_ADC();
186     batteryVoltage = ((float)batteryADC / 1023.0) * ADC_REF_VOLTAGE * VOLTAGE_DIV_RATIO;
187     batteryVoltage = constrain(batteryVoltage, 0, BAT_MAX_VOLTAGE);
188 }
189
190 void setup(){
191     Wire.begin();
192     oled.begin(&Adafruit128x64,OLED_I2C_ADDR);
193     for(int i=0;i<SERVO_TOTAL;i++){
194         servo[i].attach(servoPins[i]);
195     }
196     loadEEP();
197     initWS2812();
198     drawStatic();
199     IrReceiver.begin(RECV_PIN,ENABLE_LED_FEEDBACK);

```

```
200 Buzzer_Setup();
201 Buzzer_Alert(2,1);
202 }
203
204 void loop(){
205     if(IrReceiver.decode()){
206         auto c=IrReceiver.decodedIRData.command;
207         switch(c){
208             case 12:
209                 sel=5;
210                 Buzzer_Alert(1,1);
211                 break;
212             case 24:
213                 sel=4;
214                 Buzzer_Alert(1,1);
215                 break;
216             case 94:
217                 sel=3;
218                 Buzzer_Alert(1,1);
219                 break;
220             case 8:
221                 sel=2;
222                 Buzzer_Alert(1,1);
223                 break;
224             case 28:
225                 sel=1;
226                 Buzzer_Alert(1,1);
227                 break;
228             case 90:
229                 sel=0;
230                 Buzzer_Alert(1,1);
231                 break;
232             case 64:
233                 target[sel]=min(180,target[sel]+stepSize);
234                 break;
235             case 25:
236                 if(sel == 4 || sel == 5){
237                     target[sel]=max(80,target[sel]-stepSize);
238                 } else {
239                     target[sel]=max(0,target[sel]-stepSize);
240                 }
241                 break;
242             case 21:
243                 target[sel]=90;
244                 Buzzer_Alert(1,1);
245                 break;
246             case 22:
247                 for(int i=0;i<SERVO_TOTAL;i++){
248                     target[i]=90;
249                 }
250                 Buzzer_Alert(1,1);
251                 break;
252             case 67:
253                 saveEEP();
254                 Buzzer_Alert(1,1);
255                 break;
```



```
256     case 69:
257         stepSize=1;
258         Buzzer_Alert(1,1);
259         break;
260     case 70:
261         stepSize=3;
262         Buzzer_Alert(1,1);
263         break;
264     case 71:
265         stepSize=5;
266         Buzzer_Alert(1,1);
267         break;
268     case 68:
269         stepSize=10;
270         Buzzer_Alert(1,1);
271         break;
272     case 66:
273         wsMode=1;
274         Buzzer_Alert(1,1);
275         break;
276     case 82:
277         wsMode=2;
278         Buzzer_Alert(1,1);
279         break;
280     case 74:
281         wsMode=3;
282         Buzzer_Alert(1,1);
283         break;
284     case 13:
285         wsMode=0;
286         Buzzer_Alert(1,1);
287         break;
288     }
289     IrReceiver.resume();
290 }
291 updateServo();
292 updateOLED();
293 updateBattery();
294 updateWS2812();
295 }
```